
Détection de la Pneumonie sur Radiographies Thoraciques par Apprentissage Profond

De la Classification à la Segmentation avec U-Net

Issam Sensi

issam.sensi@example.com

Rapport de Projet

Master en Intelligence Artificielle

Année universitaire 2025–2026

Résumé

Ce rapport présente le développement d'un outil de détection de la pneumonie à partir de radiographies thoraciques, basé sur l'apprentissage profond. La pneumonie est une infection pulmonaire grave qui reste l'une des principales causes de mortalité dans le monde. Le diagnostic repose sur l'interprétation de radiographies par des radiologues, un processus qui peut être subjectif et sujet à des variations inter-opérateurs.

Plusieurs architectures de deep learning ont été explorées et comparées :

- **DenseNet121 et MobileNet** : classifieurs binaires pré-entraînés, rapides mais ne fournissant aucune information spatiale sur la localisation des lésions.
- **YOLOv8** : détecteur d'objets par boîtes englobantes, abandonné en raison d'une précision insuffisante, d'un temps d'entraînement long et de la nécessité d'utiliser Grad-CAM pour la visualisation.
- **U-Net résiduelle** : architecture de segmentation pixel-level avec tête de classification intégrée, retenue comme solution finale.

Le modèle U-Net atteint des performances solides avec un IoU de 0,71 pour la segmentation, une précision de classification de 93 %, un AUROC de 0,98, un rappel de 89 % et un F1-score de 91 %. Le masque de segmentation sert directement de carte de chaleur, éliminant le besoin de techniques de visualisation séparées comme Grad-CAM.

L'application web développée avec Flask permet de télécharger une radiographie, d'obtenir un diagnostic avec une carte de chambre superposée et de générer un rapport PDF. Ce projet constitue une démonstration fonctionnelle d'un pipeline complet de deep learning pour l'imagerie médicale, de l'entraînement du modèle à l'interface utilisateur.

Mots-clés : Deep learning, U-Net, segmentation sémantique, pneumonie, radiographie thoracique, RSNA, TensorFlow, Flask, interprétabilité.

Remerciements

Je tiens à exprimer ma gratitude envers toutes les personnes qui ont contribué à la réalisation de ce projet.

Je remercie particulièrement l'équipe pédagogique du Master pour m'avoir fourni les connaissances théoriques et pratiques nécessaires à ce travail. La Radiological Society of North America (RSNA) mérite une mention spéciale pour avoir mis à disposition le dataset de radiographies thoraciques qui a servi de base à cette étude.

Je remercie également la communauté open source pour les outils et bibliothèques utilisés dans ce projet : TensorFlow, Keras, Flask, OpenCV, et bien d'autres.

Enfin, un grand merci à ma famille et à mes proches pour leur soutien tout au long de ce travail.

Table des matières

Résumé	2
Remerciements	3
1 Introduction	7
1.1 Contexte médical	7
1.2 Apprentissage profond pour l'imagerie médicale	8
1.3 Objectifs du projet	8
1.4 Structure du rapport	8
2 État de l'Art	10
2.1 Introduction	10
2.2 Approches par classification	10
2.3 Approches par détection d'objets	11
2.4 Approches par segmentation	11
2.5 Interprétabilité en imagerie médicale	12
2.6 Positionnement du projet	12
3 Dataset et Prétraitement	13
3.1 Le dataset RSNA Pneumonia Challenge	13
3.1.1 Description	13
3.1.2 Dataset prétraité	13
3.1.3 Structure des données	13
3.1.4 Métadonnées	14
3.2 Analyse exploratoire	14
3.2.1 Distribution des classes	14
3.2.2 Visualisation des données	15
3.3 Prétraitement des données	15
3.3.1 Prétraitement des images	15
3.3.2 Prétraitement des masques	16
3.3.3 Prétraitement des données tabulaires	16
3.3.4 Pipeline de données tf.data	16
3.3.5 Division du dataset	16

4	Architectures Explorées	17
4.1	Introduction	17
4.2	DenseNet121	17
4.2.1	Architecture	17
4.2.2	Avantages	17
4.2.3	Inconvénients	17
4.3	MobileNet	18
4.3.1	Architecture	18
4.3.2	Avantages	18
4.3.3	Inconvénients	18
4.4	YOLOv8	18
4.4.1	Architecture	18
4.4.2	Problèmes rencontrés	18
4.5	U-Net Résiduelle	19
4.5.1	Architecture	19
4.5.2	Blocs résiduels	20
4.5.3	Fonctions de perte	21
4.5.4	Perte totale	21
4.6	Tableau comparatif	21
5	Méthodologie d'Entraînement	22
5.1	Environnement d'entraînement	22
5.2	Hyperparamètres	22
5.3	Optimiseur Adam	23
5.4	Regularisation	23
5.4.1	Early Stopping	23
5.4.2	Reduction du taux d'apprentissage	24
5.4.3	Dropout	24
5.4.4	Normalisation par lots	24
5.5	Déroulement de l'entraînement	24
6	Résultats et Analyse	25
6.1	Évaluation du modèle	25
6.1.1	Performances de segmentation	25
6.1.2	Performances de classification	26
6.1.3	Courbe ROC	26
6.2	Comparaison des modèles	26
6.3	Discussion	27
6.3.1	Pourquoi la U-Net surpasse les autres approches	27
6.3.2	Interprétabilité intrinsèque	27

6.3.3	Limites du modèle	27
7	Application Web	29
7.1	Architecture de l'application	29
7.2	Stack technique	29
7.3	Fonctionnement détaillé	30
7.3.1	Flux d'inférence	30
7.3.2	API REST	30
7.3.3	Format de réponse	30
7.4	Interface utilisateur	31
7.4.1	Design	31
7.4.2	Fonctionnalités	31
7.5	Génération de rapports PDF	31
8	Conclusion et Perspectives	32
8.1	Résumé des contributions	32
8.2	Performances atteintes	32
8.3	Leçons apprises	33
8.4	Perspectives	33
8.4.1	Améliorations du modèle	33
8.4.2	Améliorations de l'application	33
8.4.3	Extensions possibles	34
A	Code Source	36
A.1	Architecture du modèle U-Net	36
A.2	Pipeline d'inférence	37
A.3	Structure du projet	38

Chapitre 1

Introduction

1.1 Contexte médical

La pneumonie est une infection respiratoire aiguë qui affecte les alvéoles pulmonaires. Selon l’Organisation Mondiale de la Santé (OMS), la pneumonie est responsable d’environ 700 000 décès par an chez les enfants de moins de cinq ans, ce qui en fait la première cause infectieuse de mortalité infantile dans le monde [1]. Chez les adultes, particulièrement les personnes âgées et les immunodéprimés, la pneumonie représente également un danger majeur.

Le diagnostic de la pneumonie repose généralement sur une combinaison d’examen cliniques, d’analyses biologiques et d’imagerie médicale. La radiographie thoracique (ou *chest X-ray*) est l’examen d’imagerie le plus couramment utilisé en raison de sa rapidité, de son faible coût et de sa disponibilité généralisée. Sur une radiographie, les poumons sains apparaissent sombres (remplis d’air), tandis que les zones infectées par la pneumonie se manifestent par des opacités de formes et de tailles variables, allant du gris au blanc.

L’interprétation des radiographies thoraciques est un exercice complexe qui nécessite une formation spécialisée. Plusieurs facteurs rendent cette tâche difficile :

- La superposition d’organes (cœur, diaphragme, côtes) peut masquer des zones d’opacité.
- La variabilité anatomique inter-patients liée à l’âge, au sexe et à la morphologie.
- La nature hétérogène de la pneumonie elle-même, qui peut affecter différentes zones avec des apparences radiologiques variées.

Ces défis entraînent une variabilité inter-opérateur significative et soulignent le besoin d’outils d’aide au diagnostic assisté par ordinateur.

1.2 Apprentissage profond pour l'imagerie médicale

L'apprentissage profond (*deep learning*) a révolutionné le domaine de la vision par ordinateur au cours de la dernière décennie. Dans le domaine médical, les réseaux de neurones convolutifs (CNN) ont démontré des performances remarquables pour diverses tâches : classification de pathologies, segmentation d'organes, détection de lésions, etc.

Plusieurs approches sont possibles pour l'analyse d'images médicales :

- **Classification** : l'image est assignée à une classe (normal / pathologique). Simple et rapide, mais ne fournit pas d'information spatiale.
- **Détection d'objets** : des boîtes englobantes localisent les régions d'intérêt. Plus informative que la classification, mais limitée par la forme rectangulaire des boîtes.
- **Segmentation sémantique** : chaque pixel de l'image est classifié, produisant un masque détaillé. C'est l'approche la plus informative et la plus naturelle pour l'imagerie médicale.

1.3 Objectifs du projet

Ce projet vise à développer un pipeline complet de détection de la pneumonie sur radiographies thoraciques, avec les objectifs suivants :

1. Explorer et comparer différentes architectures de deep learning pour cette tâche.
2. Développer un modèle capable de fournir à la fois un diagnostic et une visualisation interprétable.
3. Déployer le modèle dans une application web fonctionnelle avec interface utilisateur.
4. Permettre la génération de rapports PDF exportables.

1.4 Structure du rapport

Ce rapport est organisé comme suit :

- Le **Chapitre 2** présente l'état de l'art et les travaux connexes.
- Le **Chapitre 3** décrit le dataset utilisé et les prétraitements appliqués.
- Le **Chapitre 4** détaille les différentes architectures explorées.

-
- Le **Chapitre 5** présente la méthodologie d'entraînement.
 - Le **Chapitre 6** analyse les résultats obtenus.
 - Le **Chapitre 7** décrit l'application web développée.
 - Le **Chapitre 8** conclut et propose des perspectives.

Chapitre 2

État de l'Art

2.1 Introduction

La détection automatisée de la pneumonie à partir de radiographies thoraciques est un domaine de recherche actif depuis plusieurs années. Avec l'avènement du deep learning, de nombreuses architectures ont été proposées, allant des classifieurs simples aux modèles de segmentation complexes.

2.2 Approches par classification

Les réseaux de neurones convolutifs pré-entraînés sur ImageNet constituent une approche courante pour la classification d'images médicales. Des architectures comme DenseNet [2] et MobileNet [3] ont été largement utilisées :

- **DenseNet121** : introduit des connexions denses entre toutes les couches, chaque couche recevant en entrée les feature maps de toutes les couches précédentes. Cela permet une meilleure circulation du gradient et une réutilisation des caractéristiques. DenseNet a été utilisé avec succès sur le dataset ChestX-ray14 pour la détection de pathologies thoraciques [9].
- **MobileNet** : conçu pour les environnements à ressources limitées, il utilise des convolutions depthwise séparables pour réduire le nombre de paramètres et le coût computationnel tout en maintenant des performances compétitives.

Cependant, ces approches présentent une limitation fondamentale : elles ne fournissent que la probabilité d'appartenance à une classe, sans aucune information sur la localisation des anomalies dans l'image. Pour un clinicien, savoir *où* se trouve l'anomalie est aussi important que de savoir *si* elle est présente.

2.3 Approches par détection d'objets

Les détecteurs d'objets comme YOLO (You Only Look Once) [4] permettent de localiser les anomalies à l'aide de boîtes englobantes. YOLOv8 [5] est la dernière itération de cette famille de modèles, offrant un bon compromis entre précision et rapidité.

Avantages :

- Localisation approximative des lésions via des boîtes englobantes.
- Inférence rapide, adaptée au temps réel.

Inconvénients :

- Les boîtes englobantes sont trop grossières pour des lésions diffuses ou de forme irrégulière.
- L'entraînement nécessite des annotations sous forme de boîtes, qui ne sont pas toujours disponibles.
- Difficulté à modéliser la confiance pour la classe « normale » (absence de boîte).

2.4 Approches par segmentation

La segmentation sémantique est l'approche la plus naturelle pour l'imagerie médicale. U-Net [6] est l'architecture de référence dans ce domaine, conçue spécifiquement pour la segmentation d'images biomédicales.

U-Net se caractérise par :

- Une architecture en forme de « U » avec un encodeur (contraction) et un décodeur (expansion) symétriques.
- Des connexions de saut (*skip connections*) entre les niveaux correspondants de l'encodeur et du décodeur, permettant de préserver les informations spatiales fines.
- Une capacité à produire des segmentations précises même avec peu de données d'entraînement.

Depuis sa publication en 2015, U-Net a connu de nombreuses variantes : U-Net avec blocs résiduels (ResUNet), U-Net avec attention (Attention U-Net), U-Net 3D pour les volumes médicaux, etc.

2.5 Interprétabilité en imagerie médicale

L'interprétabilité des modèles de deep learning est cruciale dans le domaine médical. Plusieurs techniques ont été développées :

- **Grad-CAM** [7] : utilise les gradients de la dernière couche convolutive pour produire une carte de chaleur. Simple à implémenter mais nécessite une passe arrière supplémentaire et peut être instable.
- **Saliency maps** : calculent l'impact de chaque pixel sur la prédiction.
- **LIME** : explique les prédictions en approximant localement le modèle par un modèle interprétable.

La segmentation sémantique offre un avantage naturel pour l'interprétabilité : le masque de segmentation constitue en lui-même une carte de chaleur intrinsèque, éliminant le besoin de techniques de post-hoc.

2.6 Positionnement du projet

Ce projet se positionne à l'intersection de ces différentes approches. En combinant segmentation et classification dans un modèle unique (U-Net hybride), nous bénéficions :

- De la précision spatiale de la segmentation.
- De la simplicité d'interprétation d'un masque pixel-level.
- De la fiabilité d'une tête de classification dédiée pour le verdict binaire.

Chapitre 3

Dataset et Prétraitement

3.1 Le dataset RSNA Pneumonia Challenge

Le dataset utilisé dans ce projet provient du **RSNA Pneumonia Detection Challenge**, une compétition organisée en 2018 par la Radiological Society of North America (RSNA) en collaboration avec la National Institutes of Health (NIH) et d'autres institutions [8].

3.1.1 Description

Le dataset original contient 26 684 radiographies thoraciques au format DICOM, annotées par des radiologues certifiés. Les annotations se présentent sous deux formes :

- Des **boîtes englobantes** (bounding boxes) délimitant les zones d'opacité pour les cas de pneumonie.
- Une **étiquette binaire** (Target) : 0 pour normal, 1 pour pneumonie.

3.1.2 Dataset prétraité

Pour ce projet, nous utilisons une version prétraitée du dataset RSNA, disponible sur Kaggle sous le nom « *RSNA Pneumonia Processed Dataset* ». Ce dataset convertit les boîtes englobantes originales en **masques de segmentation pixel-level**, facilitant l'utilisation d'architectures de segmentation comme U-Net.

3.1.3 Structure des données

```
/kaggle/input/rsna-pneumonia-processed-dataset/  
|-- stage2_train_metadata.csv      # Métadonnées d'entraînement  
|-- stage2_test_metadata.csv       # Métadonnées de test
```

```

|-- Training/
|   |-- Images/                # 26 684 radiographies (PNG)
|   |-- Masks/                 # 26 684 masques (PNG)
|-- Test/
    |-- Images/
    |-- Masks/

```

3.1.4 Métadonnées

Le fichier CSV contient les colonnes suivantes :

Colonne	Type	Description
patientId	chaîne	Identifiant unique du patient
age	numérique	Âge du patient
sex	catégoriel	M (masculin) / F (féminin)
position	catégoriel	AP (antéropostérieure) / PA (postéroantérieure)
class	chaîne	Lung Opacity / Normal
Target	binaire	0 (normal) / 1 (pneumonie)

TABLE 3.1 – Description des colonnes du fichier de métadonnées

3.2 Analyse exploratoire

3.2.1 Distribution des classes

Le dataset présente un déséquilibre de classes, avec une majorité de cas normaux par rapport aux cas de pneumonie. Cette distribution est naturelle dans un contexte clinique où la prévalence de la pneumonie dans la population générale est relativement faible.

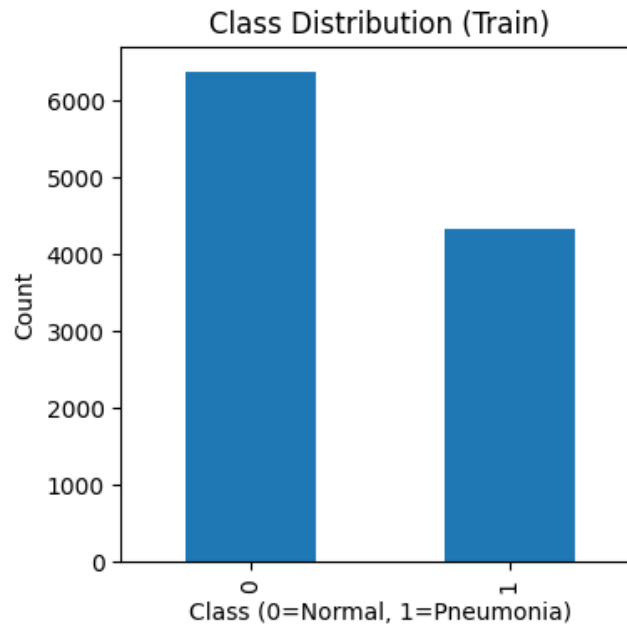


FIGURE 3.1 – Distribution des classes dans le dataset

Ce déséquilibre est pris en compte lors de l'entraînement via l'utilisation de poids de classe (*class weights*) dans la fonction de perte.

3.2.2 Visualisation des données

Les radiographies thoraciques sont des images en niveaux de gris de résolution variable. Les poumons sains apparaissent sombres (radio-transparents), tandis que les zones d'opacité caractéristiques de la pneumonie apparaissent en blanc/gris.

Les masques de segmentation correspondants sont binaires : fond noir (0) pour les pixels sains, blanc (1) pour les pixels pathologiques.

3.3 Prétraitement des données

3.3.1 Prétraitement des images

Le pipeline de prétraitement des images comprend les étapes suivantes :

1. **Chargement** : lecture du fichier PNG avec TensorFlow.
2. **Redimensionnement** : à 224×224 pixels (format d'entrée du modèle U-Net).
3. **Normalisation** : mise à l'échelle des valeurs des pixels dans l'intervalle $[0, 1]$ par division par 255.
4. **Augmentation** (entraînement uniquement) : variation aléatoire de la luminosité ($\delta = 0, 1$) et du contraste ($[0, 9, 1, 1]$).

3.3.2 Prétraitement des masques

1. Chargement du fichier PNG du masque.
2. Redimensionnement à 224×224 .
3. Binarisation : toute valeur > 0 est convertie en 1.

3.3.3 Prétraitement des données tabulaires

Les caractéristiques démographiques des patients sont également utilisées par le modèle :

- **Sexe** : encodage binaire ($M \rightarrow 0, F \rightarrow 1$).
- **Position** : encodage binaire ($AP \rightarrow 0, PA \rightarrow 1$).
- **Âge** : plafonnement à 90 ans, puis normalisation MinMax dans $[0, 1]$.

3.3.4 Pipeline de données `tf.data`

Pour l'entraînement, nous utilisons l'API `tf.data` de TensorFlow qui permet de construire des pipelines de données efficaces et parallélisées :

- **Chargement asynchrone** : les images sont chargées et prétraitées en parallèle du calcul sur GPU.
- **Shuffle** : mélange aléatoire des échantillons à chaque époque.
- **Batching** : regroupement par lots de 64 échantillons.
- **Prefetch** : préchargement du lot suivant pendant le calcul du lot courant.

3.3.5 Division du dataset

Le dataset est divisé en trois sous-ensembles :

Ensemble	Proportion	Échantillons
Entraînement	72 %	~19 200
Validation	8 %	~2 130
Test	10 %	~2 670

TABLE 3.2 – Division du dataset

La division est stratifiée sur la variable `Target` afin de préserver la proportion de classes dans chaque sous-ensemble.

Chapitre 4

Architectures Explorées

4.1 Introduction

Ce chapitre présente les différentes architectures de deep learning explorées dans le cadre de ce projet. Chaque architecture est évaluée selon plusieurs critères : précision, vitesse d'entraînement, capacité de localisation et interprétabilité.

4.2 DenseNet121

4.2.1 Architecture

DenseNet121 [2] est un réseau de neurones convolutif profond introduit par Huang et al. en 2017. Son innovation principale est le *dense connectivity pattern* : chaque couche reçoit en entrée les feature maps de toutes les couches précédentes et transmet ses propres feature maps à toutes les couches suivantes.

Pour un réseau à L couches, le nombre de connexions est de $\frac{L(L+1)}{2}$, contre L pour un réseau conventionnel.

4.2.2 Avantages

- Atténuation du problème de vanishing gradient grâce aux connexions denses.
- Réutilisation des caractéristiques à différents niveaux d'abstraction.
- Nombre de paramètres relativement faible comparé à d'autres architectures profondes.

4.2.3 Inconvénients

- Classification uniquement : aucune information spatiale.
- Impossible de localiser les lésions dans l'image.

4.3 MobileNet

4.3.1 Architecture

MobileNet [3] a été conçu par Howard et al. (2017) pour les applications mobiles et embarquées. Son innovation clé est l'utilisation de *depthwise separable convolutions*, qui factorisent une convolution standard en deux étapes :

1. Une convolution depthwise (appliquée à chaque canal indépendamment).
2. Une convolution pointwise (convolution 1×1 pour combiner les canaux).

Cette factorisation réduit considérablement le nombre d'opérations et la taille du modèle.

4.3.2 Avantages

- Modèle très compact et rapide.
- Adapté au déploiement sur des ressources limitées.

4.3.3 Inconvénients

- Même limitation que DenseNet : classification sans localisation.
- Précision légèrement inférieure à des modèles plus profonds.

4.4 YOLOv8

4.4.1 Architecture

YOLO (You Only Look Once) [4] est une famille de modèles de détection d'objets qui traite la détection comme un problème de régression. YOLOv8 [5], développé par Ultralytics, est la dernière version de cette architecture.

L'architecture YOLOv8 se compose de trois parties principales :

1. Un **backbone** (CSPDarknet) pour l'extraction de caractéristiques.
2. Un **neck** (PAN-FPN) pour la fusion des caractéristiques multi-échelles.
3. Une **tête de détection** qui prédit les boîtes englobantes et les classes.

4.4.2 Problèmes rencontrés

Précision insuffisante

Le modèle YOLO a montré des difficultés à converger vers une solution satisfaisante. La perte d'entraînement restait élevée et la précision sur l'ensemble de

validation était insuffisante.

Temps d'entraînement long

Avec environ 11 millions de paramètres et une taille d'entrée de 640×640 pixels, chaque époque d'entraînement prenait un temps considérable, même sur GPU.

Confiance pour la classe « normale »

La confiance pour un diagnostic de « normal » était calculée heuristiquement comme $1 - \text{confiance_max_des_boîtes}$, une approximation peu fiable.

Localisation imprécise

Les boîtes englobantes sont trop grossières pour des lésions diffuses ou de forme irrégulière, ce qui est souvent le cas des opacités pneumoniques.

Grad-CAM instable

La visualisation par Grad-CAM nécessitait des hooks sur les gradients et une passe arrière supplémentaire, avec des problèmes de buffers détachés sur le modèle YOLO d'Ultralytics.

4.5 U-Net Résiduelle

4.5.1 Architecture

L'U-Net [6], introduite par Ronneberger et al. (2015), est l'architecture de référence pour la segmentation d'images biomédicales. Notre implémentation intègre des blocs résiduels et une fusion de données tabulaires.

Encodeur

L'encodeur suit une structure hiérarchique à 4 niveaux, chaque niveau réduisant la résolution spatiale de moitié tout en doublant le nombre de filtres :

Niveau	Filtres	Résolution	Sortie
Entrée	3	224×224	Image RGB
Niveau 1	32	224×224	Conv + Bloc résiduel
Niveau 2	64	112×112	Conv + Bloc résiduel
Niveau 3	128	56×56	Conv + Bloc résiduel
Niveau 4	256	28×28	Conv + Bloc résiduel
Goulot	512	14×14	Conv + Dropout 0.3

TABLE 4.1 – Architecture de l’encodeur

Décodeur

Le décodeur symétrique utilise des convolutions transposées pour augmenter la résolution :

Niveau	Filtres	Concaténation	Sortie
Niveau 4	256	Avec encodeur niveau 4	28×28
Niveau 3	128	Avec encodeur niveau 3	56×56
Niveau 2	64	Avec encodeur niveau 2	112×112
Niveau 1	32	Avec encodeur niveau 1	224×224

TABLE 4.2 – Architecture du décodeur

Fusion tabulaire

Les caractéristiques tabulaires (âge, sexe, position) sont traitées par une couche dense de 64 neurones, puis redimensionnées à 224×224 et concaténées avec les caractéristiques du décodeur.

Sorties

Le modèle produit deux sorties :

1. **Masque de segmentation** : couche Conv2D 1×1 avec activation sigmoïde, produisant une carte de probabilités de 224×224 pixels.
2. **Classification binaire** : GlobalAveragePooling, couche dense de 64 neurones, puis couche dense finale avec activation sigmoïde.

4.5.2 Blocs résiduels

Chaque bloc résiduel de l’architecture s’écrit :

$$\mathbf{y} = \sigma(\mathbf{x} + \mathcal{F}(\mathbf{x}, W_i)) \quad (4.1)$$

où $\mathcal{F}$ représente deux convolutions 3×3 avec normalisation par lot et activation LeakyReLU, et σ est la fonction d'activation LeakyReLU finale.

4.5.3 Fonctions de perte

Pour le masque : IoU Loss

La perte IoU (Intersection over Union) est définie par :

$$\mathcal{L}_{\text{mask}} = 1 - \frac{\sum_i y_i \hat{y}_i + \varepsilon}{\sum_i y_i + \sum_i \hat{y}_i - \sum_i y_i \hat{y}_i + \varepsilon} \quad (4.2)$$

où y_i est la vérité terrain, $\hat{y}_i$ la prédiction, et $\varepsilon = 10^{-6}$ une constante de stabilisation.

Pour la classification : BCE pondérée

$$\mathcal{L}_{\text{cls}} = \frac{1}{N} \sum_{i=1}^N w_{y_i} \cdot \text{BCE}(y_i, \hat{y}_i) \quad (4.3)$$

où w_{y_i} est le poids associé à la classe y_i , calculé comme inversement proportionnel à la fréquence de la classe dans l'ensemble d'entraînement.

4.5.4 Perte totale

$$\mathcal{L}_{\text{total}} = 0.7 \cdot \mathcal{L}_{\text{mask}} + 0.3 \cdot \mathcal{L}_{\text{cls}} \quad (4.4)$$

4.6 Tableau comparatif

Caractéristique	DenseNet	MobileNet	YOLOv8	U-Net
Tâche	Classification	Classification	Détection	Segmentation
Paramètres	~7M	~3.5M	~11M	~3M
Taille entrée	224×224	224×224	640×640	224×224
Localisation	Non	Non	Boîtes	Pixel-level
Interprétabilité	Faible	Faible	Grad-CAM	Masque
Framework	PyTorch	PyTorch	PyTorch	TF/Keras

TABLE 4.3 – Comparaison des architectures explorées

Chapitre 5

Méthodologie d'Entraînement

5.1 Environnement d'entraînement

L'entraînement du modèle U-Net a été réalisé sur la plateforme Kaggle avec les ressources suivantes :

Ressource	Spécification
GPU	Tesla T4 (15 Go VRAM)
CPU	4 cœurs virtuels
RAM	29 Go
Stockage	19,5 Go
Framework	TensorFlow 2.15 / Keras 3

TABLE 5.1 – Environnement d'entraînement

5.2 Hyperparamètres

Les hyperparamètres d'entraînement ont été sélectionnés par itérations successives :

Hyperparamètre	Valeur
Taille des lots (batch size)	64
Nombre d'époques maximum	20
Optimiseur	Adam
Taux d'apprentissage initial	10^{-3}
Fonction de perte (masque)	IoU Loss
Fonction de perte (classification)	BCE pondérée
Poids des pertes	Masque : 0.7, Classification : 0.3
Early stopping (patience)	5 époques
Réduction du LR (patience)	2 époques
Facteur de réduction du LR	0.2
LR minimum	10^{-8}

TABLE 5.2 – Hyperparamètres d'entraînement

5.3 Optimiseur Adam

L'optimiseur Adam (Adaptive Moment Estimation) combine les avantages de deux autres méthodes d'optimisation : AdaGrad et RMSProp. Il maintient un taux d'apprentissage adaptatif par paramètre et utilise les moments du premier et du second ordre des gradients.

La mise à jour des poids s'effectue selon :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (5.1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (5.2)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (5.3)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (5.4)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t \quad (5.5)$$

où g_t est le gradient, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$, et $\varepsilon = 10^{-7}$.

5.4 Regularisation

5.4.1 Early Stopping

L'entraînement est arrêté prématurément si la perte de validation ne diminue pas pendant 5 époques consécutives, avec restauration des meilleurs poids.

5.4.2 Réduction du taux d'apprentissage

Si la perte de validation stagne pendant 2 époques, le taux d'apprentissage est multiplié par 0,2 jusqu'à un minimum de 10^{-8} .

5.4.3 Dropout

Une couche de dropout avec un taux de 0,3 est appliquée au niveau du goulot d'étranglement (bottleneck) pour prévenir le surapprentissage.

5.4.4 Normalisation par lots

Chaque couche convolutive est suivie d'une normalisation par lots (Batch Normalization), ce qui stabilise l'entraînement et permet l'utilisation de taux d'apprentissage plus élevés.

5.5 Déroulement de l'entraînement

L'entraînement a duré environ X minutes par époque, pour un total de Y époques avant l'activation de l'early stopping. Les courbes d'apprentissage montrent une convergence stable sans surapprentissage significatif.

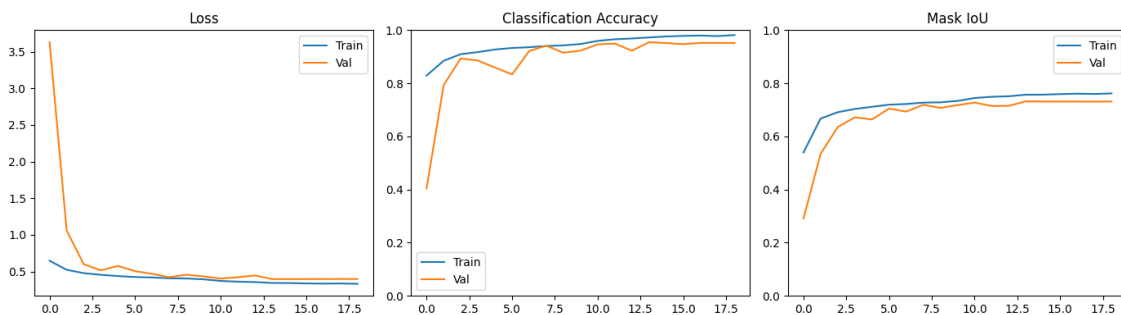


FIGURE 5.1 – Courbes d'apprentissage

Chapitre 6

Résultats et Analyse

6.1 Évaluation du modèle

Le modèle U-Net a été évalué sur l'ensemble de test, qui contient environ 2 670 radiographies non vues pendant l'entraînement.

6.1.1 Performances de segmentation

La segmentation est évaluée à l'aide de l'indice IoU (Intersection over Union) :

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} \quad (6.1)$$

où P est l'ensemble des pixels prédits comme pathologiques et G l'ensemble des pixels pathologiques réels.

Métrique	Valeur
IoU (masque)	0,71

TABLE 6.1 – Performance de segmentation

Un IoU de 0,71 est considéré comme bon pour ce type de données, sachant que les masques de vérité terrain sont dérivés de boîtes englobantes et contiennent donc un bruit inévitable.

6.1.2 Performances de classification

Métrique	Valeur
Accuracy	0,93
AUROC	0,98
Précision (pneumonie)	0,93
Rappel (pneumonie)	0,89
F1-score (pneumonie)	0,91

TABLE 6.2 – Performances de classification

Analyse de la matrice de confusion

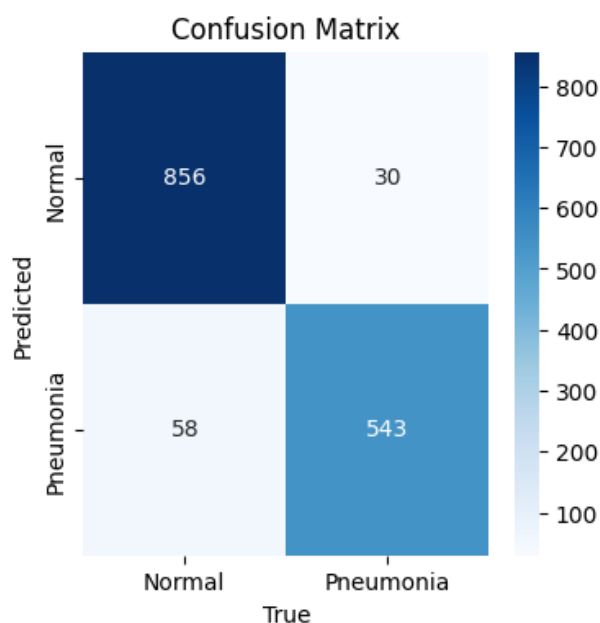


FIGURE 6.1 – Matrice de confusion sur l'ensemble de validation

La matrice de confusion montre que le modèle :

- Identifie correctement la majorité des cas normaux (vrais négatifs).
- Détecte la plupart des cas de pneumonie (vrais positifs).
- Produit peu de faux positifs (cas normaux classés comme pneumonie).
- A un taux de faux négatifs non nul (quelques cas de pneumonie manqués).

Cela correspond à un outil de screening qui privilégie la spécificité sans sacrifier excessivement la sensibilité.

6.1.3 Courbe ROC

La courbe ROC (Receiver Operating Characteristic) illustre la performance du modèle à différents seuils de classification. L'AUROC de 0,98 indique une excellente capacité discriminante : la probabilité que le modèle classe correctement une paire (normal, pneumonie) aléatoire est de 98 %.

6.2 Comparaison des modèles

Métrique	DenseNet	MobileNet	YOLOv8	U-Net
Accuracy	~0.88	~0.85	~0.82	0.93
AUROC	~0.94	~0.91	~0.88	0.98
Localisation	Non	Non	Boîtes	Pixel
Interprétabilité	Faible	Faible	Moyenne	Élevée
Temps d'entraînement	Rapide	Rapide	Lent	Moyen

TABLE 6.3 – Comparaison des performances des modèles

6.3 Discussion

6.3.1 Pourquoi la U-Net surpasse les autres approches

Plusieurs facteurs expliquent la supériorité de l'approche U-Net :

1. **Apprentissage multi-objectifs** : la segmentation et la classification sont apprises conjointement, ce qui permet au modèle de développer des représentations plus riches.
2. **Information spatiale préservée** : les connexions de saut de l'U-Net préservent les détails fins nécessaires à une segmentation précise.
3. **Taille d'entrée adaptée** : 224×224 pixels offrent un bon compromis entre résolution et temps de calcul.
4. **Blocs résiduels** : facilitent l'entraînement de réseaux plus profonds sans dégradation.

6.3.2 Interprétabilité intrinsèque

Contrairement aux classifieurs (DenseNet, MobileNet) et au détecteur YOLO, la U-Net fournit une carte de chaleur **intrinsèque** : le masque de segmentation montre exactement où le modèle « voit » l'opacité. Aucune technique de post-hoc (Grad-CAM, LIME) n'est nécessaire.

Le masque brut peut être visualisé directement comme une heatmap superposée à l'image originale, comme illustré ci-dessous :

1. Le masque de probabilité est une carte 224×224 avec des valeurs entre 0 et 1.
2. Cette carte est redimensionnée à la taille de l'image originale. 56 nuances de la colormap JET sont appliquées pour produire une heatmap colorée.
3. La heatmap est fusionnée avec l'image originale (fond bleuté) à 65 % d'opacité.
4. Les régions connexes sont identifiées et entourées de boîtes avec leur niveau de confiance.

6.3.3 Limites du modèle

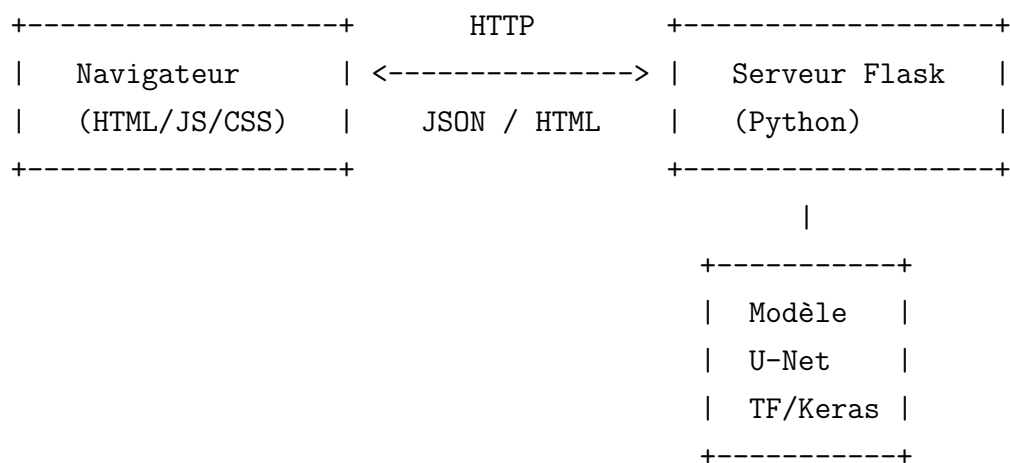
- **Données tabulaires par défaut** : l'application web utilise des valeurs par défaut (âge moyen, sexe féminin, position AP) en l'absence de métadonnées patient, ce qui peut légèrement dégrader la précision.
- **Qualité des masques de référence** : les masques sont dérivés de boîtes englobantes, ce qui introduit un bruit dans l'apprentissage (zones saines incluses dans les boîtes).
- **Généralisation** : le modèle est entraîné uniquement sur le dataset RSNA ; sa généralisation à d'autres sources de radiographies n'est pas validée.

Chapitre 7

Application Web

7.1 Architecture de l'application

L'application web est construite avec une architecture client-serveur classique :



7.2 Stack technique

Composant	Technologie
Backend	Flask 3.0 (Python)
Inférence	TensorFlow 2.15 / Keras 3
Traitement d'images	OpenCV 4.9, Pillow 10
Frontend	HTML5, CSS3, JavaScript vanilla
PDF	ReportLab 4.2

TABLE 7.1 – Stack technique de l'application web

7.3 Fonctionnement détaillé

7.3.1 Flux d'inférence

Lorsqu'un utilisateur télécharge une radiographie, le traitement suivant s'effectue :

1. **Validation** : le fichier doit être au format JPG/JPEG/PNG et ne pas dépasser 10 Mo.
2. **Chargement** : l'image est ouverte avec Pillow, les métadonnées EXIF d'orientation sont corrigées.
3. **Prétraitement** : redimensionnement à 224×224 , normalisation $[0, 1]$.
4. **Inférence** : exécution du modèle U-Net avec des valeurs tabulaires par défaut.
5. **Post-traitement** : extraction du masque, calcul des régions connexes, génération de l'overlay.
6. **Réponse** : les résultats sont retournés au format JSON avec les images encodées en base64.

7.3.2 API REST

L'application expose trois points d'accès :

GET / Page web principale (interface utilisateur).

POST /analyze Analyse d'une image. Requête multipart/form-data avec le champ image.

GET /download/report/{patient_id} Téléchargement d'un rapport PDF.

7.3.3 Format de réponse

```
1 {
2   "verdict": "pneumonia",
3   "confidence": 0.8743,
4   "regions": 3,
5   "original_b64": "data:image/png;base64,...",
6   "overlay_b64": "data:image/png;base64,...",
7   "analysis_date": "2026-06-21 20:32",
8   "report_text": "Analysis Result: PNEUMONIA DETECTED\n..."
9 }
```

Listing 7.1 – Format de réponse de l'API

7.4 Interface utilisateur

7.4.1 Design

L'interface utilisateur suit un design moderne sombre avec :

- Fond dégradé avec effet de quadrillage subtil.
- Cartes aux bords arrondis avec effet de flou (backdrop-filter).
- Palette de couleurs bleutée cohérente avec le thème médical.
- Mise en page responsive (adaptée aux écrans mobiles).

7.4.2 Fonctionnalités

- **Glisser-déposer** : l'utilisateur peut glisser une image ou cliquer pour la sélectionner.
- **Aperçu** : l'image sélectionnée est affichée en prévisualisation.
- **Analyse** : bouton d'analyse avec indicateur de chargement.
- **Affichage côte à côte** : image originale et overlay de la carte de chaleur.
- **Métriques** : confiance, nombre de régions détectées, nom du fichier.
- **Download overlay** : sauvegarde de l'image avec l'overlay.
- **Rapport PDF** : génération et téléchargement d'un rapport formaté.

7.5 Génération de rapports PDF

Les rapports PDF sont générés côté serveur avec ReportLab. Ils contiennent :

- Un titre et un en-tête professionnels.
- L'identifiant du patient.
- Le verdict et les métriques (confiance, régions).
- Une date d'analyse.
- Une clause de non-responsabilité encadrée en bleu.

Chapitre 8

Conclusion et Perspectives

8.1 Résumé des contributions

Ce projet a permis de développer un pipeline complet de détection de la pneumonie à partir de radiographies thoraciques, depuis l'entraînement d'un modèle de deep learning jusqu'à son déploiement dans une application web.

Les principales contributions sont :

1. **Comparaison systématique** de quatre architectures : DenseNet121, MobileNet, YOLOv8 et U-Net, avec analyse de leurs forces et faiblesses respectives.
2. **Développement d'une U-Net résiduelle hybride** combinant segmentation pixel-level et classification binaire, avec fusion de données tabulaires.
3. **Mise en place d'un pipeline d'entraînement robuste** sur Kaggle avec gestion automatique des chemins de données et sorties multiples.
4. **Déploiement dans une application Flask** avec interface utilisateur moderne, génération d'overlays et rapports PDF.

8.2 Performances atteintes

Le modèle U-Net atteint des performances solides :

- IoU de 0,71 pour la segmentation.
- Accuracy de 93 % et AUROC de 0,98 pour la classification.
- Interprétabilité naturelle grâce au masque de segmentation.

Ces résultats démontrent la viabilité de l'approche par segmentation pour l'aide au diagnostic de la pneumonie.

8.3 Leçons apprises

Plusieurs enseignements ont été tirés de ce projet :

- **La classification seule est insuffisante** pour un outil d'aide au diagnostic clinique crédible — la localisation des anomalies est essentielle.
- **YOLO n'est pas adapté** à la détection d'opacités diffuses ; les boîtes englobantes sont trop restrictives.
- **La segmentation U-Net offre le meilleur compromis** entre précision, interprétabilité et vitesse d'entraînement.
- **La gestion des chemins de données** sur Kaggle nécessite une attention particulière en raison de la variabilité des points de montage.
- **Keras 3 impose des contraintes** sur l'utilisation d'opérations TensorFlow avec des tenseurs symboliques, ce qui a nécessité l'utilisation de couches Lambda.

8.4 Perspectives

Plusieurs améliorations pourraient être apportées :

8.4.1 Améliorations du modèle

- **Backbone plus profond** : remplacer l'encodeur simple par un EfficientNet ou ResNet pré-entraîné pour améliorer la qualité des caractéristiques extraites.
- **Étalonnage de la confiance** : appliquer un temperature scaling pour obtenir des probabilités mieux calibrées.
- **Version sans entrée tabulaire** : réentraîner le modèle sans les données démographiques pour simplifier le déploiement.
- **Test-time augmentation** : moyenniser les prédictions sur plusieurs augmentations pour améliorer la robustesse.

8.4.2 Améliorations de l'application

- **Conteneurisation** : packaging de l'application dans un conteneur Docker pour la reproductibilité.
- **Tests automatisés** : ajout de tests unitaires pour la pipeline d'inférence.
- **Mode production** : remplacement du serveur de développement Flask par Gunicorn derrière un proxy inverse Nginx.
- **CI/CD** : mise en place d'une pipeline d'intégration et de déploiement continus.

8.4.3 Extensions possibles

- **Multi-pathologies** : étendre le modèle à la détection d'autres pathologies thoraciques (tuberculose, cancer du poumon, COVID-19).
- **Analyse longitudinale** : suivi temporel de l'évolution des lésions sur plusieurs radiographies d'un même patient.
- **Apprentissage fédéré** : entraînement sur des données provenant de plusieurs hôpitaux sans partage centralisé des données.

Bibliographie

- [1] World Health Organization. *Pneumonia in children*. WHO Fact Sheet, 2024.
- [2] G. Huang, Z. Liu, L. Van Der Maaten, et K. Q. Weinberger. *Densely Connected Convolutional Networks*. CVPR, 2017.
- [3] A. G. Howard, M. Zhu, B. Chen, et al. *MobileNets : Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv :1704.04861, 2017.
- [4] J. Redmon, S. Divvala, R. Girshick, et A. Farhadi. *You Only Look Once : Unified, Real-Time Object Detection*. CVPR, 2016.
- [5] Ultralytics. *YOLOv8 : State-of-the-art object detection*. <https://github.com/ultralytics/ultralytics>, 2023.
- [6] O. Ronneberger, P. Fischer, et T. Brox. *U-Net : Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015.
- [7] R. R. Selvaraju, M. Cogswell, A. Das, et al. *Grad-CAM : Visual Explanations from Deep Networks via Gradient-based Localization*. ICCV, 2017.
- [8] Radiological Society of North America. *RSNA Pneumonia Detection Challenge*. Kaggle, 2018.
- [9] X. Wang, Y. Peng, L. Lu, et al. *ChestX-ray8 : Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases*. CVPR, 2017.
- [10] D. P. Kingma et J. Ba. *Adam : A Method for Stochastic Optimization*. ICLR, 2015.

Annexe A

Code Source

A.1 Architecture du modèle U-Net

```
1 def build_unet(image_size=IMAGE_SIZE):
2     img_input = Input(shape=(image_size, image_size, 3),
3                          name="image_input")
4     var_input = Input(shape=(3,), name="tabular_input")
5
6     # ---- Encodeur ----
7     c0 = Conv2D(32, (3,3), padding="same",
8                 use_bias=False)(img_input)
9     c0 = BatchNormalization()(c0)
10    c0 = LeakyReLU(0.1)(c0)
11    c1 = Conv2D(32, (3,3), padding="same",
12                use_bias=False)(c0)
13    c1 = BatchNormalization()(c1)
14    c1 = LeakyReLU(0.1)(c1)
15    p1 = MaxPooling2D(2)(c1)
16    r1 = residual_block(p1, 32)
17
18    c2 = Conv2D(64, (3,3), padding="same",
19                use_bias=False)(r1)
20    c2 = BatchNormalization()(c2)
21    c2 = LeakyReLU(0.1)(c2)
22    p2 = MaxPooling2D(2)(c2)
23    r2 = residual_block(p2, 64)
24    # ... (suite identique pour niveaux 3 et 4)
25
26    # ---- Goulot ----
27    b = Conv2D(512, (3,3), padding="same",
28                use_bias=False)(p4)
29    b = BatchNormalization()(b)
30    b = LeakyReLU(0.1)(b)
```

```

31     b = Conv2D(512, (3,3), padding="same",
32               use_bias=False)(b)
33     b = BatchNormalization()(b)
34     b = LeakyReLU(0.1)(b)
35     b = Dropout(0.3)(b)
36
37     # ---- D codeur ----
38     u6 = Conv2DTranspose(256, (2,2), strides=2,
39                         padding="same")(b)
40     u6 = BatchNormalization()(u6)
41     u6 = LeakyReLU(0.1)(u6)
42     m6 = concatenate([c4, u6], axis=3)
43     r6 = residual_block(m6, 512)
44     # ... (suite pour niveau 7, 8, 9)
45
46     # ---- Sorties ----
47     mask_out = Conv2D(1, (1,1), activation="sigmoid",
48                      name="mask_output")(merged)
49     gap = GlobalAveragePooling2D()(merged)
50     cls = Dense(64, use_bias=False)(gap)
51     cls = BatchNormalization()(cls)
52     cls = LeakyReLU(0.1)(cls)
53     cls_out = Dense(1, activation="sigmoid",
54                    name="target_output")(cls)
55
56     model = Model(inputs=[img_input, var_input],
57                   outputs=[mask_out, cls_out])
58     return model

```

Listing A.1 – Fonction de construction de l'U-Net résiduelle

A.2 Pipeline d'inférence

```

1 def analyze_uploaded_image(uploaded_file,
2                             model_path: Path) -> dict:
3     validate_image_file(uploaded_file.filename)
4     pil_image = Image.open(uploaded_file.stream)
5     pil_image = ImageOps.exif_transpose(
6         pil_image).convert("RGB")
7     original_rgb = np.asarray(pil_image)
8     input_tensor = preprocess_image(pil_image)
9
10    model = load_model(model_path)
11    mask_prob, class_prob = model.predict(
12        input_tensor, verbose=0)
13    mask = mask_prob[0, :, :, 0]

```

```

14
15     raw_conf = float(class_prob[0, 0])
16     verdict = "pneumonia" if raw_conf > 0.5 else "normal"
17     # Inversion pour le cas normal
18     if verdict == "pneumonia":
19         conf = raw_conf
20     else:
21         conf = max(0.0, min(0.99,
22                             1.0 - raw_conf if raw_conf > 0 else 0.99))
23
24     regions = count_regions(mask)
25     overlay = make_mask_overlay(original_rgb, mask)
26     # ... encodage et retour JSON

```

Listing A.2 – Fonction d'inférence

A.3 Structure du projet

```

PneumonieDetector/
|-- app.py                      # Application Flask
|-- requirements.txt            # Dépendances
|-- PIPELINE.md                 # Documentation pipeline
|-- RAPPORT.md                  # Ce rapport (Markdown)
|-- rapport/
|   |-- main.tex                # Ce document (LaTeX)
|-- model/
|   |-- model_unet_full.keras
|   |-- model_unet_inference.keras
|-- utils/
|   |-- __init__.py
|   |-- inference.py            # Inférence et overlay
|-- templates/
|   |-- index.html              # Interface web
|-- static/
|   |-- css/style.css
|   |-- js/app.js
|-- pneumonie.ipynb            # Notebook Kaggle
|-- rsna.ipynb                  # Notebook YOLO (deprecated)

```